We need to build a working MVP of our college project "AI Employee Operating System (AI-EOS)" RIGHT NOW.

I have very limited time. Do NOT just give me explanations, pseudocode, or a plan. ACTUALLY CREATE THE PROJECT, WRITE THE CODE, INSTALL REQUIRED DEPENDENCIES, RUN IT, DEBUG errors, and leave me with a working application that I can demonstrate tomorrow.

I have attached our project presentation. Use it as the reference for the project's intended architecture and terminology.

==========
PROJECT
==========

AI Employee Operating System (AI-EOS)

The idea is a multi-agent AI workforce for business operations.

For the MVP, we need ONE complete working business workflow:

Business Event
↓
Manager / Orchestrator Agent
↓
Inventory Agent
↓
Procurement Agent
↓
Finance Agent
↓
Risk & Approval Engine
↓
Execution
↓
Audit Log

The goal is NOT to finish the entire production system tonight.

The goal is to make this CORE MULTI-AGENT PIPELINE genuinely work end-to-end.

==========
DEMO SCENARIO
==========

Use this business scenario:

A company's product inventory has fallen below the minimum required level.

Example:

Product: Laptop Stand
Current Stock: 12
Minimum Required Stock: 50

Therefore:

Quantity to order = 50 - 12 = 38

Procurement Agent should select a supplier from MOCK supplier data.

Example:

ABC Supplies
Unit Price = ₹850

38 × ₹850 = ₹32,300

Finance Agent should check this against a MOCK budget.

Example:

Available Budget = ₹100,000

Then Risk Engine evaluates the purchase.

RULES:

If purchase amount < ₹50,000:
    LOW RISK
    AUTO EXECUTE

If purchase amount >= ₹50,000:
    HIGH RISK
    HUMAN APPROVAL REQUIRED

For high-risk transactions, the workflow must pause and show an Approve / Reject option.

After approval, the system should continue execution.

Execution should be simulated. DO NOT make real purchases or call real financial APIs.

Generate a fake Purchase Order such as:

PO-2026-001

Finally, create an audit log containing all agent actions.

==========
TECHNOLOGY
==========

Use:

- Python
- FastAPI
- LangGraph
- LangChain
- An LLM API
- React or Next.js for frontend if appropriate

Use the simplest reliable setup.

IMPORTANT:
If an API key is required, use an environment variable such as:

OPENAI_API_KEY=

Do NOT hardcode API keys.

If the LLM API is unavailable, create a clean fallback/mock mode so that the complete workflow can STILL be demonstrated locally.

==========
NO DATABASE FOR THIS MVP
==========

DO NOT USE:

- PostgreSQL
- MySQL
- MongoDB
- Redis

Use LangGraph/Python in-memory state for now.

The state should contain information such as:

- business_event
- product
- current_stock
- minimum_stock
- inventory_result
- procurement_result
- finance_result
- risk_result
- approval_status
- execution_result
- audit_log

The database will be added later.

==========
AGENTS
==========

Implement actual specialized agents.

1. MANAGER AGENT

Role:
- Acts as orchestrator.
- Receives the business event.
- Starts the appropriate workflow.
- Passes information through the workflow.

2. INVENTORY AGENT

Role:
- Analyze inventory.
- Determine whether stock is below threshold.
- Calculate required quantity.

Example output:

```
{
  "status": "LOW_STOCK",
  "current_stock": 12,
  "required_stock": 50,
  "quantity_to_order": 38
```

}

3. PROCUREMENT AGENT

Role:
- Receive inventory result.
- Check mock supplier data.
- Select a supplier.
- Calculate total purchase amount.

Example:

```
{
  "supplier": "ABC Supplies",
  "unit_price": 850,
  "quantity": 38,
  "total_cost": 32300
}
```

4. FINANCE AGENT

Role:
- Receive procurement result.
- Check available budget.
- Determine whether the purchase is within budget.

Example:

```
{
  "available_budget": 100000,
  "purchase_amount": 32300,
  "status": "APPROVED"
}
```

5. RISK / APPROVAL ENGINE

Do NOT let the LLM randomly decide financial risk.

Use deterministic Python rules.

< ₹50,000:
    LOW
    auto execution

>= ₹50,000:
    HIGH
    human approval required

6. EXECUTION

Simulate creation of a purchase order.

Example:

```
{
  "po_id": "PO-2026-001",
  "status": "CREATED"
}
```

7. AUDIT LOGGER

Every agent should add an entry to the audit log.

Example:

Manager Agent → Business event received
Inventory Agent → Low stock detected
Procurement Agent → Supplier selected
Finance Agent → Budget approved
Risk Engine → LOW RISK
Execution → PO created

==========
LANGGRAPH
==========

Use LangGraph for the orchestration.

The graph should actually contain nodes for the agents.

Conceptually:

START
 ↓
manager
 ↓
inventory
 ↓
procurement
 ↓
finance
 ↓
risk
 ↓
conditional branch
  ├—— low risk → execution
  └—— high risk → human approval → execution
 ↓
audit
 ↓
END

Use shared LangGraph state so that information produced by one agent is available to the next agent.

DO NOT create four unrelated chatbot endpoints.

The agents must actually collaborate through shared workflow state.

==========
FRONTEND
==========

After the backend works, create a SIMPLE dashboard.

Do not spend time making a fancy UI.

It should contain:

AI EMPLOYEE OPERATING SYSTEM

Business Event

Product:
Laptop Stand

Current Stock:
12

Minimum Stock:
50

[RUN WORKFLOW]

Agent Workflow

✓ Manager Agent
↓
✓ Inventory Agent
↓
✓ Procurement Agent
↓
✓ Finance Agent
↓
✓ Risk Engine
↓
✓ Execution
↓
✓ Audit

Results:

Inventory:
38 units required

Supplier:
ABC Supplies

Purchase:
₹32,300

Finance:
APPROVED

Risk:
LOW

Execution:
PO-2026-001 CREATED

Audit Log

...

For high-risk transactions, display:

HUMAN APPROVAL REQUIRED

Purchase Amount: ₹80,000
Risk: HIGH

[APPROVE] [REJECT]

The Approve button should resume the workflow.

==========
TWO DEMO CASES
==========

Make sure BOTH work.

CASE 1 — LOW RISK

Product:
Laptop Stand

Current Stock:
12

Minimum:
50

Supplier price:
₹850

Total:
₹32,300

Expected:

LOW RISK
↓
AUTO EXECUTE
↓
PO CREATED
↓
AUDIT LOG

CASE 2 — HIGH RISK

Create another test case where the purchase amount is above ₹50,000.

Expected:

HIGH RISK
↓
HUMAN APPROVAL REQUIRED
↓
Approve / Reject

↓
If approved:
PO CREATED
↓
AUDIT LOG

==========
FASTAPI
==========

Expose a backend endpoint such as:

POST /api/workflow/run

It should accept the business event and start the workflow.

Create appropriate endpoints for the approval flow if needed.

The frontend should communicate with the FastAPI backend.

==========
PROJECT STRUCTURE
==========

Create a clean structure similar to:

```
ai-eos/
 │
 ├── backend/
 │    ├── main.py
 │    ├── requirements.txt
 │    │
 │    ├── agents/
 │    │    ├── manager.py
 │    │    ├── inventory.py
 │    │    ├── procurement.py
 │    │    ├── finance.py
 │    │    └── risk.py
 │    │
 │    ├── graph/
 │    │    ├── state.py
 │    │    └── workflow.py
 │    │
 │    ├── tools/
 │    │    ├── inventory_tool.py
 │    │    ├── supplier_tool.py
 │    │    ├── finance_tool.py
 │    │    └── purchase_tool.py
 │    │
 │    └── audit.py
 │
 └── frontend/
      └── React/Next.js application
```

You may modify this structure if there is a simpler and more reliable implementation.

==========
VERY IMPORTANT DEVELOPMENT RULES
==========

1. DO NOT only tell me how to build it.

Actually build it.

2. DO NOT stop after creating files.

Run the application.

3. If there are errors:
   - inspect the error
   - fix the code
   - run again
   - continue until working

4. Do not introduce unnecessary technologies.

5. Do not add a database.

6. Do not add AWS.

7. Do not add MCP yet.

8. Do not add authentication yet.

9. Do not add real business APIs.

10. Do not spend excessive time on UI design.

11. Prioritize a WORKING END-TO-END MULTI-AGENT PIPELINE.

12. Keep mock business data clearly separated so it can later be replaced with real tools/ database/MCP.

13. Use structured outputs wherever possible.

14. Make the system easy to extend later.

==========
SUCCESS CRITERIA
==========

Do not consider the task complete until I can run the project and demonstrate:

1. Enter/trigger a business event.

2. Manager Agent starts the workflow.

3. Inventory Agent detects low stock.

4. Procurement Agent selects a supplier.

5. Finance Agent checks the budget.

6. Risk Engine evaluates the transaction.

7. Low-risk transaction automatically executes.

8. High-risk transaction pauses for human approval.

9. Approve/Reject works.

10. A simulated Purchase Order is generated.

11. An audit log displays the complete agent workflow.

12. Frontend displays the workflow and results.

==========
FINAL REQUIREMENT
==========

This is a college project prototype, not a production banking system.

Prioritize:
WORKING > SIMPLE > DEMONSTRABLE > EXTENSIBLE

Do not waste time implementing production infrastructure.

Start by inspecting the current workspace and then BUILD THE PROJECT.

If a project already exists in the workspace, use it and modify it rather than creating unnecessary duplicate projects.

After implementation, give me only:
- what you built
- how to run it
- what URLs/commands to use
- any environment variables I need
- what to click during tomorrow's demo

But first, actually BUILD AND TEST THE APPLICATION.